CS-107 Lecture 13
Assembly Language

Troy Astarte

Megan Venn-Wycherley

Recap: machine language

- **Machine language** is a set of binary coded instructions built into the hardware of a particular computer, used directly by that computer.
- **The CPU decodes the bit pattern itself.** Its digital logic recognises the binary representation of each instruction, so no translation step is involved.
- **Instruction sets are processor-specific:** each processor type can have its own set, so machine code does not transfer between families.
- **Each instruction does one low-level task,** which is why even a short program runs to many instructions.

Recap: the Pep/9 virtual computer

- **A virtual computer** is a hypothetical machine that keeps the features of a real one worth illustrating, without the rest.
- **Pep/9,** designed by Stanley Warford, has 40 machine-language instructions and a 16-bit architecture.
- **Its registers:** the program counter holds the address of the next instruction, the instruction register holds a copy of the one being executed, and the accumulator holds data and results.
- **Main memory is 64 KB,** addressed one byte at a time.

Recap: instruction format

- **An instruction has two parts:** the instruction specifier, and an operand specifier that some instructions do not need.
- **The instruction specifier splits three ways:** the operation code, the register specifier, and the addressing-mode specifier.
- **Operation code** states which instruction to carry out; the register specifier states which register to use, which for us is the accumulator.
- **The addressing-mode specifier** states how to read the operand, so the same opcode behaves differently depending on those bits.

Recap: addressing modes and I/O

- **Immediate addressing:** the operand specifier holds the value itself, so the instruction carries its own data.
- **Direct addressing:** the operand specifier holds a memory address, and the value is fetched from there.
- **Input and output are memory-mapped:** devices sit at physical memory locations, so the same load and store instructions handle them.
- **In Pep/9,** input is at FC15 and output at FC16. Loading from FC15 reads a character; storing to FC16 writes one, a single byte at a time.

Goals for Today

- What assembly language is, and how mnemonics replace binary instructions
- The assembly process, and what the assembler produces
- The Pep/9 assembly instructions and pseudo-ops
- Naming memory locations with labels instead of addresses
- Branching and loops, and what they do to the program counter
- Worked programs: printing "Hi", and adding two numbers the user enters

Intro to Assembly

Assembly languages

- **Assembly Language**

A language that uses **mnemonic** codes to represent machine-language instructions.

Much easier on the eyes (than machine code anyway)

- **Assembler**

A program that reads each of the instructions in mnemonic form and translates it into the machine-language equivalent.

Assembly languages

- Given that instructions can differ between chipsets, it is important to have a method to take our (slightly) higher level programming languages and convert them into machine code.
- It is the machine code that will be executed. The higher languages are used to provide ease to the programmer in developing more complex behaviours.
- Common assembly languages include ARM and x86

The assembly process

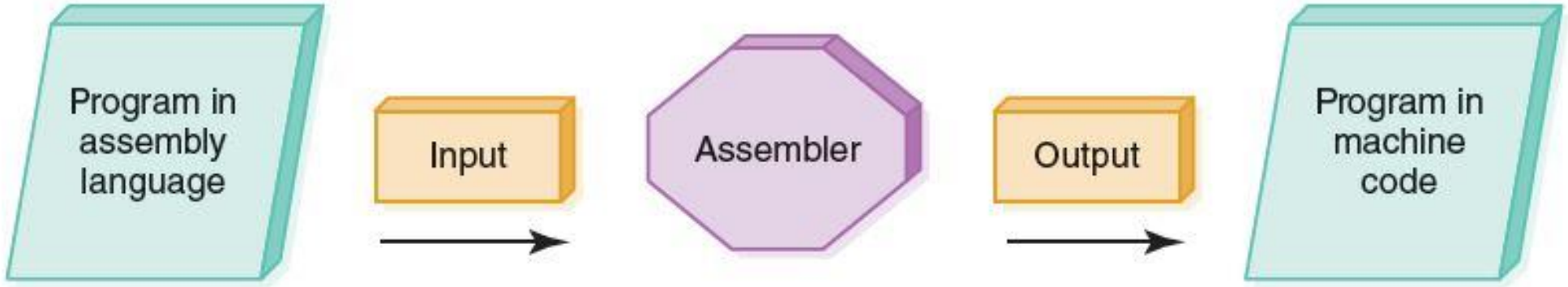
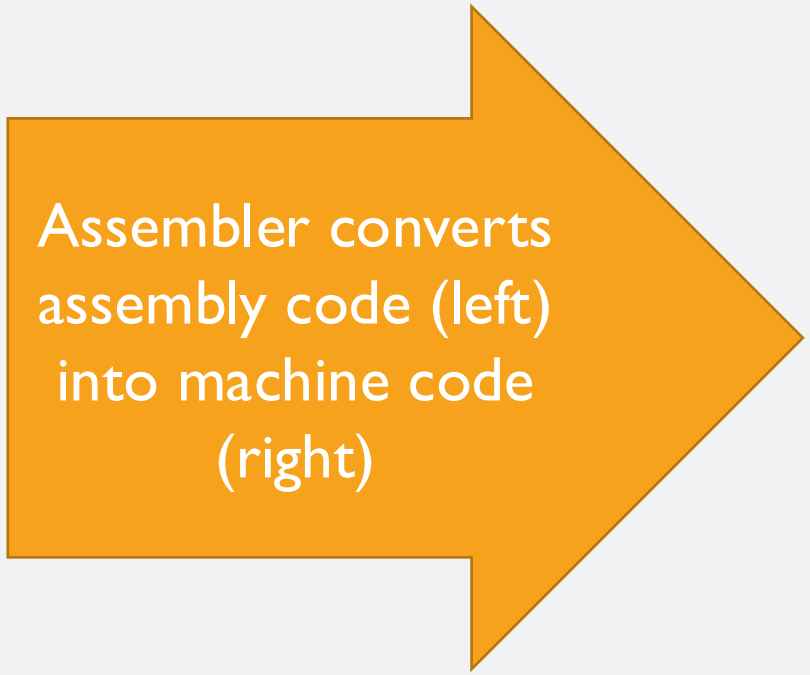


FIGURE 6.5 Assembly process

The assembler

```
LDBA 0x0048, i
STBA 0xFC16, d
LDBA 0x0065, i
STBA 0xFC16, d
LDBA 0x006C, i
STBA 0xFC16, d
LDBA 0x006C, i
STBA 0xFC16, d
LDBA 0x006F, i
...
```




```
C0 00 48 F1 FC 16
C0 00 65 F1 FC 16
C0 00 6C F1 FC 16
C0 00 6C F1 FC 16
C0 00 6F F1 FC 16
...
```

Pep/9 assembly instructions

- There are 39 assembly instructions in Pep/9
- There are also N "pseudo-ops" which act as instructions to the assembler, not as instructions for the CPU to execute.
- We will only take a look at a few of these, so that you can appreciate the purpose of assembly languages, the function of the assembler, and the behaviour when executed.

Pep/9 assembly instructions

Mnemonic	Operand, Mode	Meaning
STOP		Stop execution
LDWA	0x008B,i	Load word 008B into accumulator
LDWA	0x008B,d	Load word located at 008B into accumulator
LDBA	0x008B,i	Load byte 008B into accumulator
LDBA	0x008B,d	Load byte located at 008B into accumulator
STWA	0x008B,d	Store word from accumulator to location 008B
STBA	0x008B,d	Store byte from accumulator to location 008B
ADDA	0x008B,i	Add 008B into accumulator
ADDA	0x008B,d	Add word located at 008B into accumulator
SUBA	0x008B,i	Subtract 008B from accumulator
SUBA	0x008B,d	Subtract word located at 008B from accumulator

What do the 'I' and 'd' stand for?

Remember back to addressing modes.

Pep/9 assembly instructions

Mnemonic	Operand, Mode	Meaning
DECI	0x008B,d	Read a decimal number and store it into location 008B
DECO	0x008B,i	Write the decimal number 139 (8B in hex)
DECO	0x008B,d	Write the decimal number stored at location 008B
STRO	0x008B,d	Write the character string stored at location 008B
BR	0x001A	Branch to location 001A
BRLT	0x001A	Branch to location 001A if the accumulator is less than zero
BREQ	0x001A	Branch to location 001A if the accumulator is equal to zero
CPWA	0x008B	Compare the word stored at 008B with the accumulator

What do the 'I' and 'd' stand for?

Remember back to addressing modes.

Pep/9 addressing modes

Addressing Mode	aaa	Letters	Operand
Immediate	000	i	OprndSpec
Direct	001	d	Mem[OprndSpec]
Indirect	010	n	Mem[Mem[OprndSpec]]
Stack-relative	011	s	Mem[SP + OprndSpec]
Stack-relative deferred	100	sf	Mem[Mem[SP + OprndSpec]]
Indexed	101	x	Mem[OprndSpec + X]
Stack-indexed	110	sx	Mem[SP + OprndSpec + X]
Stack-deferred indexed	111	sfx	Mem[Mem[SP + OprndSpec] + X]

Pep/9 pseudo-ops

Pseudo-op	Operand	Meaning
.END		Signals the end of the assembly-language program
.ASCII	"banana\x00"	Represents a string of ASCII characters
.WORD	0x008B	Reserves a word in memory and stores a value in it
.BLOCK	number of bytes	Reserves a particular number of bytes in memory

Pseudo-ops are for the assembler.
They are not translated to machine code, but certain ops may result in reserved memory locations which can be used by the machine code.

Example Program

Example Program 3: Printing "Hi"

- Let's just print "Hi" for the moment. "Hello World" was a bit long.

Source Code - Hi.pep

```
LDBA    0x0048,i    ; Load 'H' into accumulator
STBA    0xFC16,d    ; Store accumulator to output device
LDBA    0x0069,i    ; Load 'i' into accumulator
STBA    0xFC16,d    ; Store accumulator to output device
STOP                ; Stop
.END
```

Example Program 3: Printing “Hi”

- The assembler takes the previous assembly code and translates it to machine code.
- It can then also be loaded into memory to be executed.
- This requires us to translate the mnemonics and their operands into hexadecimal, and to place this into memory addresses (beginning with address 0)

Example Program 3: Printing "Hi"

Source Code – Hi.pep

```
        LDBA    0x0048,i    ; Load 'H' into accumulator
        STBA    0xFC16,d    ; Store accumulator to output device
        LDBA    0x0069,i    ; Load 'i' into accumulator
        STBA    0xFC16,d    ; Store accumulator to output device
        STOP                ; Stop
        .END
```

Assembler Listing – Hi.pepl

```
-------------------------------------------------------------------------------
      Object
Addr  code    Symbol    Mnemon  Operand     Comment
-------------------------------------------------------------------------------
0000  D00048            LDBA    0x0048,i    ;Load 'H' into accumulator
0003  F1FC16            STBA    0xFC16,d    ;Store accumulator to output device
0006  D00069            LDBA    0x0069,i    ;Load 'i' into accumulator
0009  F1FC16            STBA    0xFC16,d    ;Store accumulator to output device
000C  00                STOP                ;Stop
000D                    .END
-------------------------------------------------------------------------------
```

Naming Memory Locations

Naming memory locations

- To save having to remember the physical memory address of a specific piece of information, we can name memory addresses and then refer to them by name. (Similar to how you declare variables in other languages)
- In Pep/9 this is done by starting the line with the name and a colon operator as follows:

name: mnemonic operand, mode

Naming memory locations

- For example, if we wanted to name the address which contains the literal value 5

five: .WORD 0x0005

This will reserve a word-size of memory and place the hexadecimal value 0x0005 within it. This location will then be named "five". We can then use this name elsewhere in our code to refer to this memory address.

Note that in this example the address contains data, or a value.

Naming memory locations

- For example, if we wanted to name the address which adds 5 to the accumulator:

addfive: ADDA 0x0005, i

This will name the memory address where this instruction is placed "addfive". We can then use this name elsewhere in our code to refer to this memory address.

Note that in this example the address contains an instruction.

Naming memory locations

- We could even combine the two together:

```
addfive: ADDA five, d
```

Notice now that we use direct addressing mode, as we want to go to the address “five”, which itself contains the value 0x0005 that we wish to use.

Branching and Loops

Branching and loops

- Remember the Fetch Decode Execute cycle?
- If we have a branching behaviour in our program, what is actually happening here is our program counter is updated to point to a new memory address, rather than the next sequential address.
- In Pep/9 assembly, we can do this using instructions such as BR (branch always), BREQ (branch if A == 0), BRGT (branch if A > 0). The operand for these instructions is the memory address to go to, if the condition is true.

Branching and loops

- For example, here is a program that only prints out the user's entered value if it is greater than 0:

```
        DECI    0x0020, d
        LDWA    0x0020, d
        BRLT    end
        DECO    0x0020, d
end:    STOP
.END
```

If the value in the accumulator < 0, then the program counter is updated to point to 'end'

Otherwise it stays as it is, pointing to the next sequential address. Which is a printing instruction.

Branching and loops

- Using this branching behaviour, we can create loops:

```
loop:   DECI 0x0020, d
        STBA 0x0020, d
        BRGT loop
        STOP
.END
```

If the value in the accumulator > 0, then the program counter is updated to point to 'loop'

Otherwise it stays as it is, pointing to the next sequential address. Which is a stop instruction

Branching Examples

- Example: A program that reads in two numbers and adds them:

Source Code – AddNums.pep

```
        BR      main        ; Branch around data
sum:    .WORD   0x0000      ; Set up sum and initialize to zero
num1:   .BLOCK  2           ; Set up two byte block for num1
num2:   .BLOCK  2           ; Set up two byte block for num2

main:   LDWA    sum,d       ; Load zero into accumulator
        DECI    num1,d      ; Read and store num1
        ADDA    num1,d      ; Add num1 to accumulator
        DECI    num2,d      ; Read and store num2
        ADDA    num2,d      ; Add num2 to accumulator
        STWA    sum,d       ; Store accumulator into sum
        DECO    sum,d       ; Output sum
        STOP                ; Stop
        .END
```

Branching Examples

- Example: Same as previous, but print error if sum is negative:

Source Code – AddNums2.pep

```
        BR      main        ; Branch to main program
sum:    .WORD   0x0000      ; Set up sum and initialize to zero
num1:   .BLOCK  2           ; Set up a two byte block for num1
num2:   .BLOCK  2           ; Set up a two byte block for num2
negMsg: .ASCII  "Error\x00" ; Error message in case sum is negative

error:  STRO    negMsg,d    ; Print the error message
        BR      finish

main:   LDWA    sum,d       ; Load zero into accumulator
        DECI    num1,d      ; Read and store num1
        ADDA    num1,d      ; Add num1 to accumulator
        DECI    num2,d      ; Read and store num2
        ADDA    num2,d      ; Add num2 to accumulator
        BRLT    error       ; Branch to error if A < 0
        STWA    sum,d       ; Store accumulator into sum
        DECO    sum,d       ; Output sum
finish: STOP                ; Stop
        .END
```

Goals for Today

- What assembly language is, and how mnemonics replace binary instructions
- The assembly process, and what the assembler produces
- The Pep/9 assembly instructions and pseudo-ops
- Naming memory locations with labels instead of addresses
- Branching and loops, and what they do to the program counter
- Worked programs: printing "Hi", and adding two numbers the user enters

Intermezzo

Who's done the most cool things with computers?

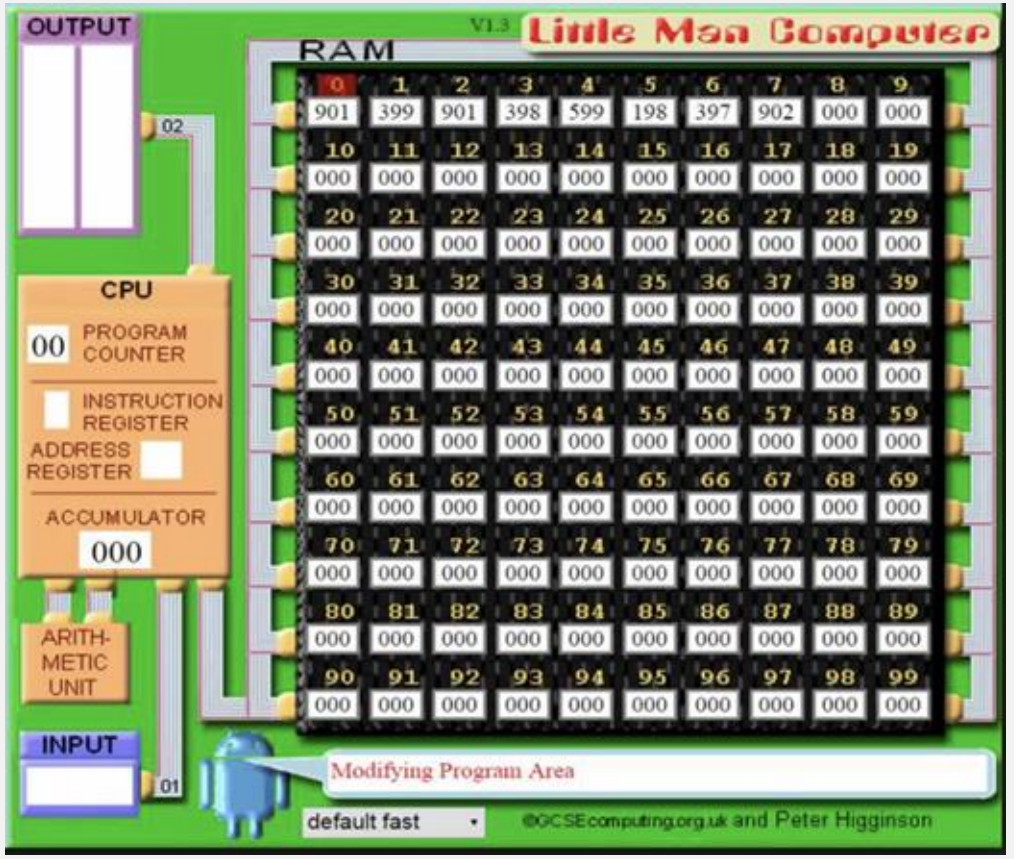
CS-107 Lecture 14
Little Man Computer

Troy Astarte

Megan Venn-Wycherley

Excursion:

Little Man Computer



The Simulator

- **Accumulator -** The active memory of the simulator. Majority of our instructions will modify contents of the accumulator.
- **Program Counter -** This shows the current memory location that the processor is running.
- **MEM Address -** The current instruction type.
- **MEM Data -** The data being used for the current instruction.

The Simulator

- **In-Box -** The input box. Enter inputs here.
- **Out-Box -** The output box. Observe outputs here.
- **CPU -** The main processing unit of the simulated machine. Carries out the fetching, decoding and executing of the instructions provided.
- **Assembly Language Code -** environment allowing instructions to be entered as LMC assembly to be loaded onto the machine.

Taking Input

Name: Input
Mnemonic: INP
Code: *901*

Description:
The input instruction takes the value in the **In-Box** and puts the value into the **Accumulator**.

Next Action:
After the value has been copied the **Program Counter** will move onto the next (sequential) memory location.

Providing Output

Name: Output
Mnemonic: OUT
Code: *902*

Description:
The output instruction takes the value in the **Accumulator** and puts the value into the **Out-Box**.

Next Action:
After the value has been copied the **Program Counter** will move onto the next (sequential) memory location.

Stopping the Program

Name: Halt
Mnemonic: HLT
Code: *000*

Description:
The halt instruction does not affect any of the memory locations and stops the program.

Next Action:
The execution of the program will stop.

Example Program 1: Specification

- Create a program which:
  - Takes in an input from the user
  - Outputs it back to the user.
- Analyse the memory locations and write down the instruction codes your program generated.

Example Program 1: LMC code

```
___    INP    ___
___    OUT    ___
___    HLT    ___
```

The first line will take an input from the user and place it in the Accumulator

The second line will take the value in the Accumulator and send it to the output.

The third line stops the program.

Storing Data

Name: Store
Mnemonic: STA *variable*
Code: *3 _ _*

Description:
The store instruction will take the data from **Accumulator** and store it into an allocated memory location which will be referred to by the variable name given.
Next Action:
After the value has been copied the **Program Counter** will move onto the next (sequential) memory location.

Retrieving Data

Name: Load
Mnemonic: LDA variable
Code: *5 _ _*

Description:
The load instruction will put the value stored at the variable location into the **Accumulator**.

Next Action:
After the value has been loaded into the **Accumulator**, the **Program Counter** will move onto the next (sequential) memory location.

Data Memory Locations

Name: Data
Mnemonic: *variable* DAT
Code: *(the data)*

Description:
The Data instruction will reserve a memory location to store data. This location can then be referred to by the given name.

Next Action:
After the memory location has been reserved, the **Program Counter** will move onto the next (sequential) memory location.

“Variable”

In the STA, LDA and DAT instructions we see the use of a “variable” argument. This refers to a named memory location.

For example:

one dat 1

will reserve a memory location, named “*one*”, containing the value 1

add one

will add the content of memory location named “one” to the accumulator.

Example Program 2: Specification

- Create a program which:
  - Takes and stores in 2 inputs from the user
  - Outputs the first input followed by the second input.

Example Program 2: LMC Code

```
(1)___    INP    ___
(2)___    STA    var
(3)___    INP    ___
(4)___    OUT    ___
(5)___    LDA    var
(6)___    OUT    ___
(7)___    HLT    ___

(8)var    DAT    ___
```

Line 1 gets the first input.

Line 2 stores this in *var*.

Line 3 gets the next input.

Line 4 prints this back out.

Line 5 loads first value back from *var*.

Line 6 prints this out.

Line 7 stops the program.

What does line 8 do?

Example Program 3: Specification

- Create a program which:
  - Takes and stores 4 inputs from the user
  - Outputs the third input to the user

This is for you to try in your own time!

Addition

Name: Addition
Mnemonic: *ADD variable*
Code: 1 _ _

Description:
The add instruction adds the value stored in the given memory location to the accumulator.

Next Action:
After the value has been loaded into the **Accumulator**, the **Program Counter** will move onto the next (sequential) memory location.

Subtraction

Name: Subtraction
Mnemonic: *SUB variable*
Code: 2 _ _

Description:
The subtraction instruction subtracts the value stored in the given memory location away from the accumulator.

Next Action:
After the value has been loaded into the **Accumulator**, the **Program Counter** will move onto the next (sequential) memory location.

Example Programs 4 and 5: Specification

- Create a program which:
  - Takes and stores in 2 inputs from the user
  - Outputs the sum of them.

- Create a program which:
  - Takes in three numbers and stores them
  - Outputs the sum of the first 2 numbers with the third subtracted. (i.e. A+B-C)

Go To (Branch Always)

Name: Branch Always
Mnemonic: *BRA variable*
Code: *6 _ _*

Description:
Moves the **Program Counter** to the memory location stored within the variable memory location.

Next Action:
After the memory location has been loaded that memory location will be executed.

Go To (Branch If Zero)

Name: Branch If Zero
Mnemonic: *BRZ variable*
Code: *7 _ _*

Description:
Moves the **Program Counter** to the memory location stored within the variable memory location if the accumulator is equal to zero.

Next Action:
After the memory location has been loaded that memory location will be executed.

Go To (Branch If Zero or Positive)

Name: Branch If Zero or Positive
Mnemonic: *BRP variable*
Code: *8 _ _*

Description:
Moves the **Program Counter** to the memory location stored within the variable memory location if the accumulator is zero or positive.

Next Action:
After the memory location has been loaded that memory location will be executed.

Little Man Computer Instruction Summary

___	**INP**	___	- Input
___	**OUT**	___	- Output
___	**HLT**	___	- Halt
___	**STA**	***var***	- *Store*
___	**LDA**	***var***	- Load
var	**DAT**	___	- Data
___	**ADD**	***var***	- Addition
___	**SUB**	***var***	- *Subtraction*
___	***BRA***	***var***	- Branch Always
___	***BRZ***	***var***	- Branch If Zero
___	***BRP***	***var***	- Branch If Positive

Example Program 6: Specification

- Create a program which:
  - Takes and stores in 2 inputs from the user.
  - Outputs the smallest number.

Hint: If you perform A - B and the result is positive, then A is **bigger** than B

Example Program 6: LMC Code

```
____    INP
____    STA    var1
____    INP
____    STA    var2

____    LDA    var1
____    SUB    var2
____    BRP    print2

____    LDA    var1
____    OUT
____    BRA    done

print2  LDA    var2
        OUT

done    HLT

var1    DAT
var2    DAT
```

Get inputs

Compute A - B and branch

Print A and branch to "done"

Print B and progress to "done"

Stop program

Reserve memory

CS-107 Lecture 14
Little Man Computer

Troy Astarte

Megan Venn-Wycherley